SOUTENANCE DE STAGE D'ÉTÉ · 2025/2026

Détection automatique et autocapture de la CIN tunisienne

Module de capture recto/verso pour Avatar-ID, application Flutter de vérification d'identité à distance

Mohammed Amine Chaieb

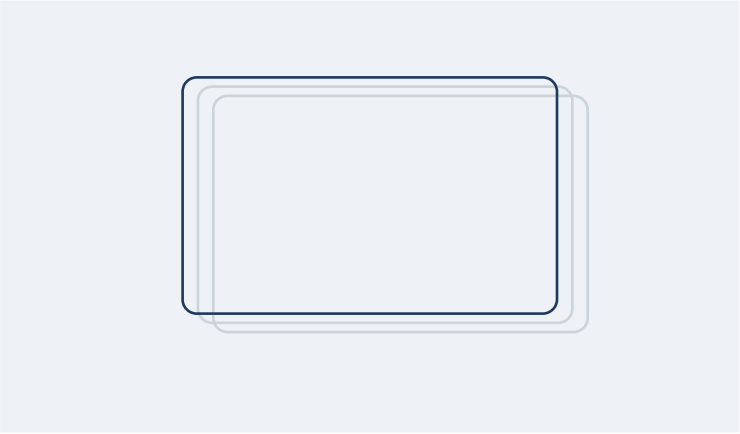
Codingaholic, La Marsa · encadrant : M. Jendoubi Mohamed Khalil

Université de la Manouba · École Nationale des Sciences de l'Informatique

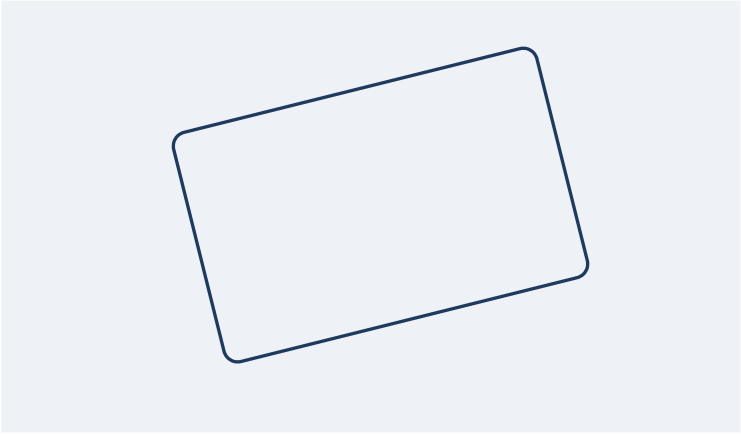


Format ID-1 · 85,6 × 54 mm · r_o **= 1,586**
les quatre coins que le module doit trouver, seul, dans le flux caméra

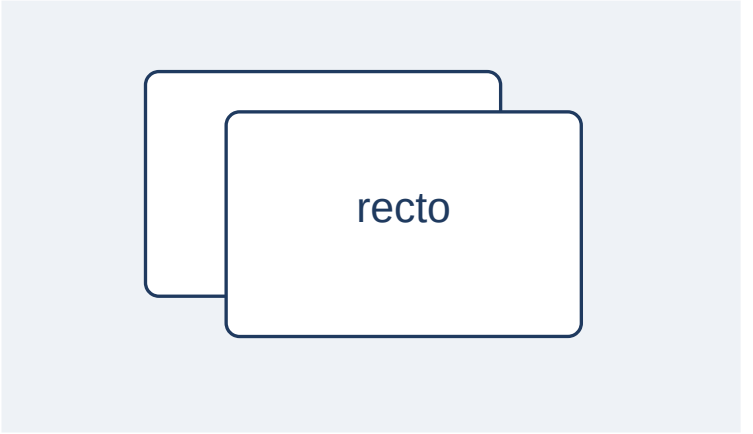
Une photo libre de CIN peut être floue, de travers ou envoyée deux fois



Floue
image inexploitable



De travers
carte mal cadrée



Deux fois le recto
verso jamais fourni

Problématique — comment détecter de façon fiable et en temps réel les contours d'une CIN dans un flux caméra mobile, quels que soient l'éclairage et l'arrière-plan ?

Avatar-ID : vérification d'identité à distance, type KYC.

Sans guidage : contrôle humain coûteux ou nouvelle saisie.

Schémas illustratifs, pas des mesures.

Cinq fonctions à automatiser, trois cibles chiffrées, deux plateformes

CE QUE LE MODULE DOIT FAIRE

1. Détecter la carte et ses contours
2. Contrôler netteté, luminosité, stabilité
3. Distinguer recto et verso, dans l'ordre
4. Capturer seul, sans bouton
5. Recadrer et améliorer l'image

Cibles, pas résultats : les mesures sont en diapositive 12.

CIBLES DU CAHIER DES CHARGES

< 100 ms de traitement par image, idéalement

≥ 90 % de recto/verso bien reconnus, en conditions normales

≈ 2 s entre carte bien positionnée et capture

Android et iOS · un seul code · six semaines

Seul OpenCV tenait Android + iOS en six semaines, sans jeu de données annoté

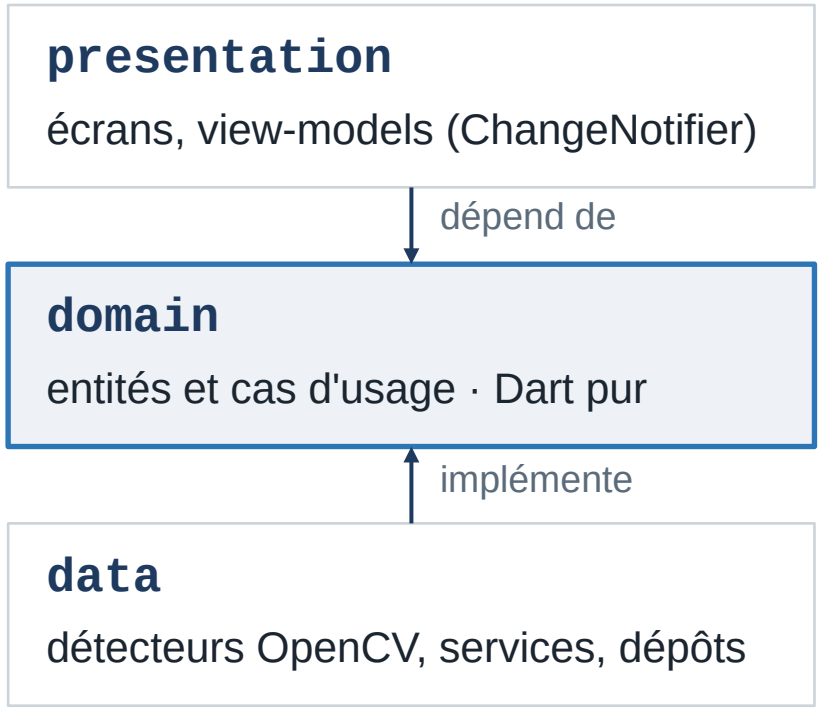
Critère	ML Kit Document Scanner	Modèle entraîné sur mesure	OpenCV (retenu)
Support iOS	**Non**	Oui, si entraîné	**Oui**
Jeu de données	Aucun	**Plusieurs centaines d'images annotées**	**Aucun**
Tenable en 6 semaines	Oui	**Non**	**Oui**
Effort de réglage	Faible	Élevé (entraînement)	**Moyen (seuils empiriques)**

En orange : le critère qui élimine l'option. Choix par élimination sur les contraintes, pas par benchmark ; Vision/VisionKit (iOS) écarté pour la même raison : deux chaînes de détection à maintenir.

Source : rapport §1.3, tableau 1.2 ; disponibilité de ML Kit Document Scanner : documentation Google ML Kit [15].

Un domaine sans Flutter ni OpenCV rend la logique testable sans caméra

UN MODULE TYPE (CLEAN ARCHITECTURE)



SEPT MODULES FEATURE-FIRST : COUCHES RÉELLEMENT PRÉSENTES

	présentation	domaine	données
`camera_capture`	●	—	—
`document_detection`	●	●	●
`image_quality`	—	—	●
`autocapture`	●	●	—
`image_postprocessing`	●	●	●
`result_preview`	●	●	—
`capture_history`	●	—	—

+ transverses : `core/services` (caméra, cascade, stockage) et `shared/utils` (conversion d'image, stabilisation)

Quatre passes de contours et un score géométrique retiennent le quadrilatère de la carte

1 — Conversion
frame YUV420 ou BGRA → Mat OpenCV

→

2 — CLAHE
contraste local renforcé

→

3 — 4 passes → masques
Canny (seuils ×0,66/×1,33 de la médiane) · seuillage adaptatif · saturation HSV + Otsu · gradient morphologique

→

4 — Score
0,8 × proportion + 0,2 × taille
retenu si ≥ 0,30

→

5 — Redressement
homographie → 856 × 540 px

CONÇU ET DÉVELOPPÉ PENDANT LE STAGE, AUTOUR DES PRIMITIVES OPENCV

Sortie anticipée
les passes suivantes ne tournent que si aucun candidat n'atteint 0,65

Suivi
carte trouvée : recherche limitée à ±20 % autour du quad précédent

Garde-fous
rejet des quads collés au bord ou couvrant plus de 75 % de l'image

Isolate Dart
pipeline hors du thread d'interface ; frame ignorée si l'isolate est occupé

primitive OpenCV — conçu dans le stage (la passe 3 combine les deux)

Sur fond clair, la détection échouait ; médiane, CLAHE et réglage des seuils l'ont corrigée

SYMPTÔME

Carte claire sur table blanche ou feuille de papier : détection très souvent en échec, alors qu'elle marchait sur fond sombre.

DEUX CAUSES

Bug : seuils de Canny calculés sur la **moyenne** et non sur la médiane : trop hauts dès que la scène est claire.

Contraste local trop faible entre la carte et le fond.

TROIS CORRECTIFS

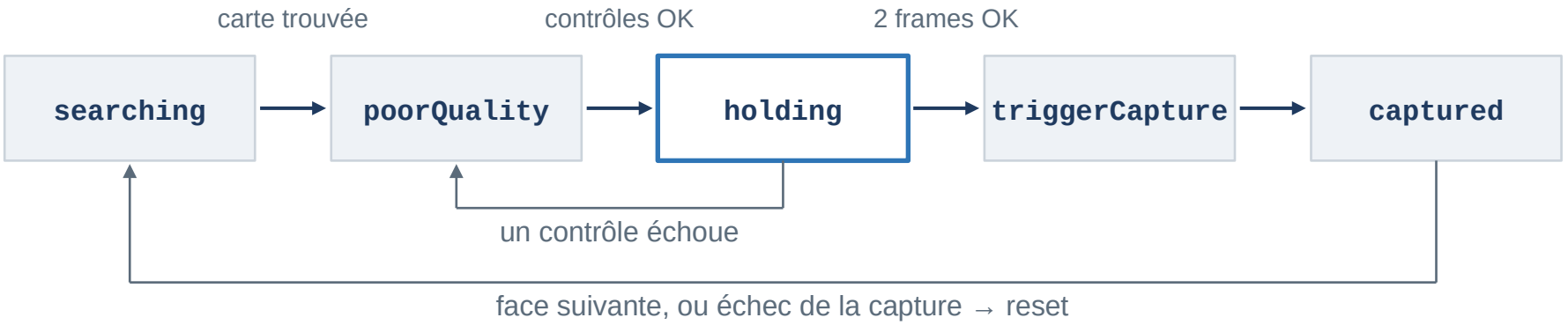
1 seuils dérivés de la vraie médiane

2 passe CLAHE avant la détection

3 réglage des seuils de score et d'aire du quadrilatère

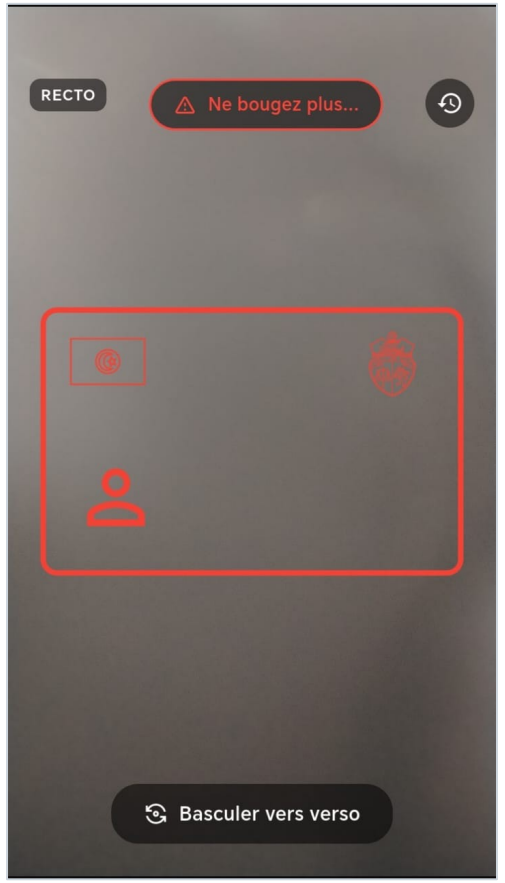
Résultat observé : taux de détection comparable sur fond clair et sur fond sombre — **gain non chiffré, sans mesure avant/après.**

La capture part seule après deux frames consécutives nettes, bien exposées et stables



Netteté	variance du Laplacien, après CLAHE, ≥ 120 (à 856 × 540 px)
Luminosité	moyenne dans [60 ; 225] et ≤ 12 % de pixels ≥ 250
Stabilité	dérive des coins ≤ 2,5 % de la diagonale, sur 3 frames
Contenu	la face attendue est reconnue (diapositive suivante)

Maintien : compteur de 2 frames (pas une minuterie). Carte perdue → retour à searching depuis tout état non terminal.



Écran réel : message de qualité « Ne bougez plus… »

Un visage et deux gabarits identifient le recto ; trois indices votent pour le verso



RECTO

Visage : cascade de Haar standard d'OpenCV, pré-entraînée (non entraînée sur la CIN), testée sur 4 rotations : elle donne aussi l'orientation.

Logo + drapeau : template matching, corrélation ≥ 0,6, CLAHE sur gabarit et région.





gabarits de référence embarqués



VERSO

Code-barres : localisé, non décodé.

Empreinte : variance du Laplacien dans [150 ; 4 000].

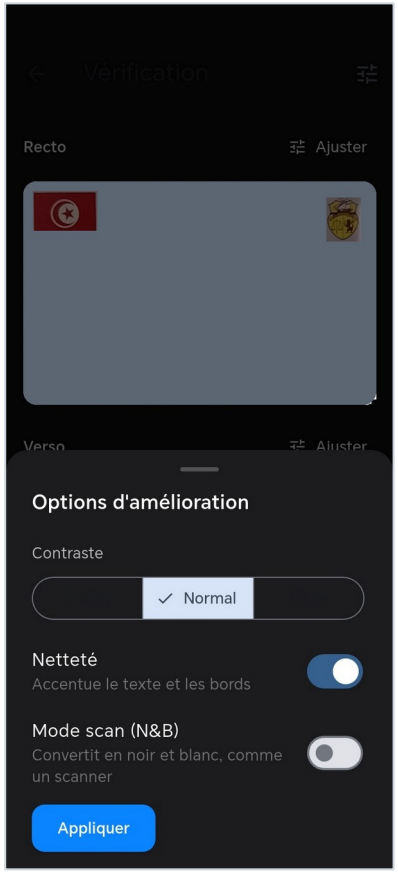
Ligne de séparation : Hough probabiliste, bas de la carte.

Vote sur 4 rotations : la mieux soutenue gagne.

Anti-doublon : après le recto, seule une face qui réunit les trois indices du verso peut déclencher la capture.

Après les deux captures, l'utilisateur vérifie, ajuste et exporte chaque face

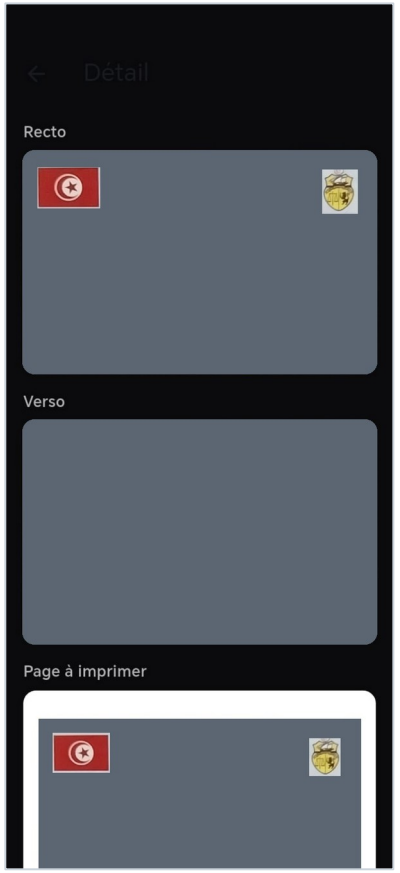
1. **Recto, puis verso**
   capturés seuls, dans l'ordre
2. **Vérification**
   contraste, netteté, mode scan (N&B)
3. **Ajustement**
   recadrage par les coins, rotation de 90°
4. **Export**
   galerie : recto, verso, page A4
5. **Historique**
   consultation, partage, suppression



Vérification, amélioration



Recadrage du verso



Page A4 à 300 dpi

177 cas de test passent, sur 21 des 28 fichiers ; les cartes réelles restent testées à la main

177 / 177

cas exécutés, 0 échec
dernière campagne flutter test (00:16)

21 / 28 fichiers exécutés

Couverture de code : non mesurée.

FICHIERS DE TEST PAR PALIER

A logique pure Dart



B plateforme simulée (mocks)



C OpenCV réel, images synthétiques



D cartes réelles, appareil : **protocole manuel, non automatisé**

Non exécutés, dont : `perspective_warper`, `blur_detector` (C), `detection_repository_impl` (B)

Les trois cibles sont atteintes de justesse, sur un échantillon encore restreint

MESURÉ	CIBLE	CE QUI EST MESURÉ	RÉSERVE
92–100 ms	< 100 ms	traitement d'une frame dans l'isolate, horodaté avant/après	borne haute = cible ; latence de bout en bout non mesurée
92 %	≥ 90 %	recto/verso reconnus, protocole manuel sur cartes réelles	nombre de cartes et d'essais non consigné dans le rapport
≈ 2 s	≈ 2 s	chronométrage manuel, de la stabilisation du quad au déclenchement	départ = stabilisation, pas la pose de la carte

Source : rapport, tableau 9.4. Conditions normales : intérieur éclairé, carte à plat, sans reflet ni angle extrême.

Cinq limites connues, à lever avant un usage en production

iOS	exigé par le cahier des charges, mais aucun essai sur iOS n'est documenté.
Échantillon	seuils réglés et mesures faites sur peu de cartes réelles.
Tests	7 fichiers non exécutés, couverture non mesurée, palier D manuel.
Timeout	le message « aucune carte détectée » après un délai n'est pas implémenté.
Anti-fraude	ni détection de vivacité ni de rejeu : le module réduit le risque, il ne le garantit pas.

Sécurité et confidentialité en détail : diapositive de secours C (rapport §7.2.3–7.2.4).

Une capture recto/verso autonome livrée en six semaines ; prochaine étape : iOS et un jeu d'images représentatif

LIVRÉ

Détection : 4 passes et un score

Autocapture : 4 contrôles, 2 frames

Recto/verso et anti-doublon

Vérification, ajustement, page A4, historique

MESURÉ

92–100 ms par frame

92 % de recto/verso reconnus

≈ 2 s jusqu'à la capture

177/177 cas sur 21 des 28 fichiers

À FAIRE

Valider sur iOS

Constituer un jeu d'images disjoint

Exécuter toute la suite, mesurer la couverture

Implémenter le timeout

Ce que j'en retiens : un choix technologique se justifie par les contraintes réelles du projet ; et une mesure ne vaut que si son protocole est consigné.

Merci de votre attention — je suis à votre disposition pour vos questions.

Tous les seuils sont empiriques ; la netteté 120 n'a de sens qu'à 856 × 540 px

Score de détection minimal	**0,30**	Netteté minimale (Laplacien)	**120**
Sortie anticipée	**0,65**	Luminosité moyenne	**[60 ; 225]**
Réduction, recherche pleine image	**0,45**	Pixels saturés tolérés (≥ 250)	**12 %**
Marge de recherche suivie	**20 %**	Dérive de coin maximale	**2,5 % diag.**
Canny bas / haut (× médiane)	**0,66 / 1,33**	Frames stables / reset	**3 / 2**
Aire de contour minimale	**3,5 %**	Frames de maintien	**2**
Aire maximale acceptée	**75 %**	Haar : échelle / voisins	**1,05 / 8**
Pénalité quad non net	**× 0,85**	Corrélation logo / drapeau	**0,6**
Proportion / taille (score)	**0,8 / 0,2**	Variance empreinte	**[150 ; 4 000]**
Redressement	**856 × 540 px**	Netteté post-traitement (a)	**0,6**

Source : rapport, tableau 5.1 ; valeurs vérifiées dans le code (lib/features/…/detectors). Réglage sur les échantillons disponibles, sans analyse de sensibilité.

177 cas exécutés ne prouvent ni la couverture ni la validité sur cartes réelles

Module	Fichiers	Exécutés	Cas
document_detection	7	4	59
image_postprocessing	7	4	24
image_quality	3	2	17
autocapture	3	3	26
result_preview	2	2	10
camera_capture	1	1	8
core/services	3	3	20
shared/utils	2	2	13
Total	**28**	**21**	**177**

Source : rapport, tableau 9.2 et §9.1. Paliers : 15 fichiers A, 6 B, 7 C ; palier D manuel.

LES 7 FICHIERS ABSENTS DU JOURNAL

- **C** perspective_warper_test
- **C** blur_detector_test
- **B** detection_repository_impl_test
- **A** detected_document_test
- **A** card_image_enhancer_test
- **A** print_page_composer_test
- **A** enhance_captured_card_test

Cinq fichiers C sur sept ont tourné : le binaire natif était disponible.

Action : flutter test --reporter expanded --coverage.

Ce que le module garantit sur la capture, et ce qu'il ne garantit pas

GARANTI PAR CONSTRUCTION

Capture en direct, depuis le flux caméra

Deux faces distinctes, dans l'ordre recto → verso

Qualité contrôlée : netteté, luminosité, stabilité

Traitement sur l'appareil, sans service tiers

NON GARANTI / NON ÉVALUÉ

Détection de vivacité, de rejeu ou de spoofing

Sort des images exportées en galerie (cloud, partage)

Historique des captures conservé sur l'appareil

Chiffrement et conformité : non évalués

Données de test : les captures montrées dans cette soutenance sont masquées dans les pixels (photo, nom, numéros, code-barres).

Source : rapport §7.2.4 et annexe C. Le §7.2.4 précise la portée de l'exigence de sécurité formulée au §1.1.2.

OpenCV fournit les primitives ; l'assemblage, les scores et la logique de capture sont développés

FOURNI (OPENCV, PLUGINS)

Canny, seuillage adaptatif, Otsu
CLAHE, morphologie
findContours, approxPolyDP
warpPerspective
Laplacien, HoughLinesP
BarcodeDetector
cascade de Haar pré-entraînée
matchTemplate
camera, gal, image

DÉVELOPPÉ PENDANT LE STAGE

conversion YUV420/BGRA → Mat
DocumentContourDetector : passes, sortie, suivi
CardQuadGeometry : score
DetectionIsolateWorker
StabilityTracker, stabilisateur
machine à états d'autocapture
BackOrientationDetector : vote
CardImageEnhancer, page A4
écrans, historique, export

RÉFÉRENCES

Canny, IEEE TPAMI, 1986
Otsu, IEEE TSMC, 1979
Zuiderveld, CLAHE, Graphics Gems IV, 1994
Viola & Jones, CVPR, 2001
Matas et al., Hough probabiliste, CVIU, 2000
Hartley & Zisserman, 2004
Martin, Clean Architecture, 2017
ISO/IEC 7810:2019 (ID-1)
Google ML Kit, Document Scanner API [15]

Sources : rapport (bibliographie, chapitres 4 à 8) et code du dépôt detection_cin.